

Codage binaire et introduction à l'assembleur

Introduction au langage assembleur

Le langage assembleur est un langage de programmation de bas niveau utilisé pour contrôler directement le matériel (processeur et mémoire). Contrairement aux langages de haut niveau comme Python ou C, chaque instruction en assembleur correspond directement à une opération du processeur.

1.1.1 Architecture du processeur

Un processeur se compose de plusieurs éléments clés :

- **Unité de commande** : Décode et enchaîne les instructions à exécuter.
- **Registres** : Mémoire interne ultra-rapide utilisée pour stocker temporairement des données.
- **ALU (Arithmetic and Logic Unit)** : Effectue les opérations arithmétiques et logiques.
- **Bus** : Permet la communication entre le processeur, la mémoire et les périphériques.

1.1.2 Registres

Les registres jouent un rôle central dans la programmation en assembleur. Ils permettent de stocker les données temporaires nécessaires aux calculs.

- **Registres généraux** : Utilisés pour les opérations courantes.
 - Exemples : R0, R1, R2, etc.
- **Registres spéciaux** : Utilisés pour des fonctions particulières (compteur de programme, pointeur de pile, résultat de comparaison, etc.).

Exemple d'utilisation d'un registre :

```
MOV R1,#5 //Charge la valeur 5 dans le registre R1
```

Remarque : il est possible de commenter son code grâce à l'ajout de //

Instructions de base

Les instructions en assembleur sont des commandes simples permettant d'effectuer des opérations sur les registres.

1.1.3 MOV : Déplacer une valeur

Syntaxe :

MOV destination, source

Cette instruction copie la valeur de la source vers la destination.

Exemple :

MOV R1, #10 ; Charge la valeur 10 dans le registre R1

MOV R2, R1 ; Copie la valeur de R1 dans R2

Après exécution, R2 contient également la valeur 10.

1.1.4 ADD : Addition

Syntaxe :

ADD destination, source1, source2

Cette instruction additionne les valeurs des registres source1 et source2, puis stocke le résultat dans destination.

Exemple :

MOV R1, #5 //Charger 5 dans R1

MOV R2, #3 //Charger 3 dans R2

ADD R3, R1, R2 //R3 = R1 + R2 (R3 contient 8)

1.1.5 SUB : Soustraction

Syntaxe :

SUB destination, source1, source2

Cette instruction soustrait source2 de source1 et stocke le résultat dans destination.

Exemple :

MOV R1, #10 //Charger 10 dans R1

MOV R2, #4 //Charger 4 dans R2

SUB R3, R1, R2 //R3 = R1 - R2 (R3 contient 6)

1.1.6 STR : Sauvegarde en mémoire RAM

Syntaxe :

STR source, destination

Place la valeur du registre source à l'adresse mémoire destination.

Exemple :

STR R1, 200 //Charger la valeur de R1 à l'adresse 200

1.1.7 LDR : Chargement depuis la mémoire RAM

Syntaxe :

LDR destination, source

Place la valeur stockée à l'adresse mémoire source dans le registre destination.

Exemple :

LDR R1, 200 //Charger la valeur à l'adresse 200 dans R1

1.1.8 CMP : Comparaison

Syntaxe :

CMP registre1, registre2

Cette instruction compare les valeurs des deux registres.

- Si registre1 > registre2, le drapeau de comparaison est mis à 1.
- Si registre1 == registre2, le drapeau de comparaison est mis à 0.

Exemple :

MOV R1, #5
MOV R2, #5
CMP R1, R2 //Compare R1 et R2

Après cette instruction, le processeur sait que R1 est égal à R2.

1.1.9 B : Saut inconditionnel

Syntaxe :

B *etiquette*

Cette instruction provoque un saut vers l'étiquette indiquée, permettant de modifier le flux d'exécution du programme.

Exemple :

B fin

Structures de contrôle

Les structures de contrôle permettent de modifier le flux d'exécution du programme en fonction de conditions ou de boucles.

1.1.10 Conditions : if/else

Les conditions sont mises en œuvre à l'aide des instructions de comparaison (CMP) et de saut conditionnel (BEQ, BGT, BLT).

Exemple : Implémentation de

if ($x > y$) { $z = x$; }

else { $z = y$; }

```
MOV R1, #10    //Charger 10 dans R1 (x)
MOV R2, #20    //Charger 20 dans R2 (y)
CMP R1, R2     //Comparer R1 et R2
BGT greater    //Sauter à greater si  $R1 > R2$ 
MOV R3, R2     //Sinon,  $R3 = R2$  ( $z = y$ )
B end          //Sauter à la fin
greater:
MOV R3, R1     // $R3 = R1$  ( $z = x$ )
end:
```

Explication :

- Si $R1 > R2$, le programme saute à l'étiquette greater et copie R1 dans R3.
- Sinon, il exécute l'instruction suivante et copie R2 dans R3.

Instruction	Signification	Condition testée
BEQ	Break if Equal	Si les deux valeurs sont égales
BNE	Break if Not Equal	Si les deux valeurs ne sont pas égales
BGT	Break if Greater	Si la première valeur est plus grande (signé)
BLT	Break if Less	Si la première valeur est plus petite (signé)

1.1.11 Boucles : for

Les boucles permettent de répéter un ensemble d'instructions tant qu'une condition est remplie.

Exemple : Implémenter une boucle for ($i = 0$; $i < 10$; $i++$)

```
MOV R1, #0    //Initialiser i à 0
loop_start:
```

```

CMP R1, #10    //Comparer i avec 10
BGT loop_end  //Sauter à la fin si i > 10
//Corps de la boucle
ADD R1, R1, #1 //Incrémenter i
B loop_start  //Retourner au début de la boucle
loop_end:

```

Explication :

- La boucle s'exécute tant que R1 est inférieur à 10.
- À chaque itération, R1 est incrémenté de 1.

1.1.12 Boucles : while

La boucle while continue d'exécuter un ensemble d'instructions tant qu'une condition est vraie.

Exemple : Implémenter une boucle while ($x < 5$)

```

MOV R1, #0    //Initialiser x à 0
while_start:
CMP R1, #5    //Comparer x avec 5
BGT while_end //Sauter à la fin si x > 5
//Corps de la boucle
ADD R1, R1, #1 //Incrémenter x
B while_start //Retourner au début de la boucle
while_end:

```

Explication :

- La boucle continue tant que R1 est strictement inférieur à 5.
- À chaque itération, R1 est incrémenté.

Exercices pratiques

1. Exercice 1 :

Écrire un programme qui place la valeur 10 dans le registre R4 et la valeur 20 dans le registre R6 puis qui va intervertir les deux valeurs.

2. Exercice 2 :

Écrire un programme assembleur qui compare deux nombres et stocke le plus grand dans un registre. Dans le cas où les nombres sont identiques il stockera le double de la valeur dans le registre.

3. Exercice 3 :

Implémenter une boucle qui affiche les nombres de 1 à 20.

4. Exercice 4 :

Écrire une boucle qui double une valeur initiale jusqu'à ce qu'elle dépasse 100.

5. Exercice 5 :

Afficher les 8 premières puissances de 2.

6. Exercice 6 :

Écrire un programme qui trouve et enregistre dans R0 le chiffre le plus grand parmi 5 chiffres donnés dans les registres de R1 à R5.

